

Spoof-Detection — Kujur et al. (2024) INS monitor with Section-5 recovery

MATLAB implementation of the CPI + solution-separation spoofing monitor from Kujur, Khanafseh & Pervan, *Optimal INS Monitor for GNSS Spoofer Tracking Error Detection*, NAVIGATION 71(1) 2024 (navi.629), with the post-detection recovery FSM (NOMINAL -> COAST -> PROBATION -> handback), packaged for a two-rate host: a 100 Hz navigation loop and a 2 Hz GNSS/monitor task.

Runtime path (what the target simulation needs)

Host contract: docs/HOST_2HZ_WIRING.m. The gate works in update-increment space ($x+ - x_bar$), so it is independent of the host's feedback/gain bookkeeping of `KF.states`.

file	role
SPF_gate.m	persistent-state 2 Hz gate; wraps SPF_protectedNav
SPF_protectedNav.m	FSM in increment form; host owns its filters, gate returns corrections
SPF_monitorPool.m	overlapping-window bank (N slots)
SPF_cpiMonitor.m, SPF_ssMonitor.m	per-window CPI (Eq. 33/35) and SS (Eq. 49-52) tests
SPF_insCoast.m, SPF_revalidation.m	INS-only propagation (E28/E31), chi-square re-validation
matrixInv.m	STAND-IN for the simulation's SVD inverse (<code>[inv, invalid] = matrixInv(A)</code>), used by the CPI and re-validation on the zero-padded covariances; replace with the simulation's own file
STRUCT_SPF.m	all struct constructors + window open/close
CST_spfParam.m	solved monitor constants (single source of truth at runtime)
CST_spfMode.m	mode enumeration
CST_gnssHybrid.m	stand-in (<code>N0_STATES = 60</code>); the simulation has its own

Harness / offline design (not deployed)

run_recovery.m (flat 2 Hz demo + offline design), run_scheduler_test.m + scheduler_sim.m (100 Hz + 2 Hz split-rate architecture test), generate_test_data.m, build_Phi_Q.m, kalman_update_step.m, kujur_params.m, solve_N_min.m, compute_PMD_eq38.m, recovery_nav_sim.m (a host following docs/HOST_2HZ_WIRING.m), ins_100hz_template.m (what the host's 100 Hz function must add).

Tests

Run runAllTests from the repo root before integrating (MATLAB; for Octave see tools/octave_shim/README.md). Do not add tools/octave_shim to a MATLAB path. It runs the four scripts below in order and prints one PASS/FAIL line each.

- `run_scheduler_test.m` — 100 Hz + 2 Hz architecture (T1-T6)
- `tests/test_monitors.m` — engine unit + Monte-Carlo checks (replaces the dual_monitor-era `test_all.m`, which cannot drive `SPF_cpiMonitor` since the window length / thresholds moved into `CST_spfParam`)
- `tests/test_variable_numMeas.m` — varying satellite count, padded vs exact
- `tests/test_feedback_split.m` — host setKF feedback split (GAIN), drain in COAST
- `tests/test_error_handlers.m` — unusable covariances via `matrixInv`, `numMeas` clamp, NaN input, anchor stamp
- `tests/test_matrixInv.m` — contract of the host's SVD inverse: zero padding, accuracy on code+carrier S, non-finite flag, rank deficiency; run it with the simulation's own `matrixInv` on the path
- `tests/test_shadow_mode.m` — guide Step 5: shadow mode on clean data (zero alarms), then a scripted attack with authority (latch, vetoes, one commit)
- `run_recovery.m` — end-to-end demo with offline design and plots

Host decisions

`docs/HOST_DECISIONS.md` tabulates, per mode and event, what `setKF` sets (`stateFB`, `KF.states`, `KF.covariance`), how the trial is handled, what each mode compares against, and the gate outputs. `docs/HOST_2HZ_WIRING.m` is the code-level contract.

Logging and diagnostics

`tools/SPF_logInit.m`, `tools/SPF_logAppend.m` and `tools/SPF_plotLog.m` (MATLAB desktop, not code generation) record the gate telemetry once per 2 Hz epoch and draw the standard figures: mode timeline with events, SS/CPI margins per axis (alarm at 1), protection level, host NIS, re-validation statistic against its threshold, dwell, coast time, faults. The header of `SPF_plotLog.m` says how to read them for a jamming or false-latch investigation. The mode figure shades the warm-up: after a gate initialisation the monitors stay disarmed (`info.armed = false`, no latch possible) until the filter has had `ARM_EPOCHS` consecutive epochs with enough rows and a protection level below `ARM_PL_MAX` (`CST_spfParam`).

Printable copies

`docs/pdf/` holds PDF renderings of this README, `docs/HOST_DECISIONS.md`, `docs/REVIEW_FINDINGS.md` and `docs/CONCEPTS.md` (separation, coast, `P_C` / `P_KF`, anchor and warm-up explained in question-and-answer form), plus `ALL_DOCS.pdf` with all of them in one file. Regenerate after editing the Markdown with `docsToPdf` (a MATLAB function in `tools/`): it writes the HTML files itself and calls Edge or Chrome headless for the PDF step; without a browser the HTML files can be printed from any browser.

Code generation

The runtime path (`SPF_gate` and everything it calls) is written for MATLAB Coder: fixed-size padded buffers, one struct layout per type from `STRUCT_SPF`, `uint8` enumeration modes, no cells, `varargin`, dynamic fields, function handles or `try/catch`. `tools/codegenSpoofMonitor.m` (MATLAB only) runs `coder.screener` on the call tree and builds a C library with the entry-point types taken from the `STRUCT_SPF` zero constructors. The gate must only run in navigation mode (see the alignment section of `docs/HOST_2HZ_WIRING.m`).

Regression

`reference/` holds the console output of both scripts from the author's MATLAB run; the current engine reproduces it exactly on MATLAB. `tools/octave_shim/` lets everything run under GNU Octave (event epochs differ by the RNG stream).

Host integration decisions

Companion to `H0ST_2HZ_WIRING.m`. Records what the host's 2 Hz function does with the operational KF, the `trialKF` and the gate outputs in every case, and what each gate mode compares against. `GAIN` is the host's constant feedback gain (0.4). "Mode" is the gate mode **after** this epoch's call to `SPF_gate`; "in probation" is the mode **before** it.

1. Definitions

Symbol	Meaning
<code>xPrior</code>	<code>activeKF.states</code> captured before calling <code>kfUpdate</code> (extrapolated at 100 Hz)
<code>xPost</code>	<code>kfUpdate</code> output: <code>activeKF.states + K*y</code>
<code>PPrior</code> , <code>PPost</code>	<code>activeKF.covariance</code> before, <code>kfUpdate</code> covariance after
<code>y</code> , <code>H</code> , <code>R</code> , <code>numMeas</code>	innovation <code>z - h(xPrior)</code> , Jacobian, measurement noise, valid rows as the filter used them (pressure row included; <code>numMeas = 1</code> on a GNSS outage)
increment	<code>xPost - xPrior = K*y</code> : the only thing the monitors consume
separation	sum of Phi-propagated increments since a reference epoch = host solution minus the coast of that reference
anchor	last alarm-free closed window: <code>separation</code> (kept live) and coast covariance
<code>probSep</code>	sum of the trial's increments since probation opened
<code>nav.state</code> , <code>nav.covar</code>	a complete <code>(x+, P+)</code> for the operational KF, same definition as the <code>kfUpdate</code> outputs
active filter	operational KF in NOMINAL and COAST, <code>trialKF</code> in PROBATION

The gate never stores an absolute state. Every quantity it keeps is a difference between two solutions that share the same mechanization history, which is what makes it invariant to how `setKF` splits `xPost` between `stateFB` and `KF.states` (verified: `tests/test_feedback_split.m`, identical estimate for `GAIN` 0.1, 0.4 and 1.0).

2. Operational KF: what `setKF` sets in every case

Exactly one `setKF` call per epoch on the operational KF. `stateFB` rows are position/velocity/attitude; bias and clock feedback follow the existing cumulative rule in every case.

Case	Gate flags	<code>stateFB</code>	<code>KF.states</code>	<code>KF.covariance</code>	Scratch <code>xPost</code> , <code>PPost</code>
NOMINAL, normal update	none	<code>GAIN * xPost</code>	<code>xPost - GAIN * xPost</code>	<code>PPost</code>	used
NOMINAL, no satellite	none	zero except biases	<code>xPost</code> except pressure-alt <code>(1-GAIN)*xPost</code>	<code>PPost</code>	used (existing rule)
NOMINAL, invalid update	none	zero	zero	as today	to the gate: <code>numMeas = 0</code> , <code>xPost = xPrior</code> , <code>PPost = PPrior</code>

Case	Gate flags	stateFB	KF.states	KF.covariance	Scratch xPost, PPost
LATCH (NOMINAL -> COAST)	nav.applyCorrection, info.eventLatched	GAIN * nav.state	nav.state - GAIN * nav.state	nav.covar (anchor coast covariance)	discarded
COAST (stays)	none	GAIN * states_extrap (drain)	states_extrap - GAIN * states_extrap	input KF.covariance unchanged (extrapolated)	discarded; only y, H, R, numMeas feed the gate
COAST -> PROBATION	kfCommand.startTrial, info.eventProbationStarted	drain, as COAST	drain, as COAST	input unchanged	discarded; next epoch trialKF = KF (states and covariance, already extrapolated)
PROBATION (stays)	none	drain, as COAST	drain, as COAST	input unchanged	belongs to the trial (see section 3)
VETO (PROBATION -> COAST)	info.eventProbationVetoed	drain, as COAST	drain, as COAST	input unchanged	trial discarded
COMMIT (PROBATION -> NOMINAL)	nav.applyCorrection, info.eventHandback	GAIN * nav.state	nav.state - GAIN * nav.state	nav.covar (trial PPost)	nav.state is the trial's xPost
No usable anchor at alarm	info.anchorMissing, no correction	as COAST	as COAST	input unchanged	discarded
Alignment (navActive = false)	gate called, returns zeroTel and re-arms itself	as today	as today	as today	as today
GNSS outage in navigation (pressure row only)	gate called with numMeas = 1; cannot pass re- validation (REVAL_MIN_MEAS = 4)	as today (no-satellite case)	as today	as today	used

Implementation: setKF itself is unchanged. The host selects its input before the one call per epoch: nav.state / nav.covar on latch and commit, the extrapolated KF.states / KF.covariance on COAST and PROBATION (the normal case on the prior is the drain), kfPost otherwise. On the two overriding branches the fields that select the case (failed, satellite count) are forced to the normal case, so a failed or no-satellite scratch update can neither zero the clean state nor trigger the 100 Hz reset. Rules behind the table:

- Whatever is fed to the mechanization is removed from KF.states in the same call. The drain applies the normal split to the extrapolated state, so the leftover of the latch correction reaches the output smoothly, by (1-GAIN) per epoch, instead of sitting in KF.states for the whole coast.
- The covariance in COAST and PROBATION is never the scratch PPost (it was reduced by a measurement being refused) and never the gate's output (only meaningful on latch and commit epochs; equal to the extrapolated covariance otherwise).
- The coast is pure INS for now: no baro update in COAST or PROBATION. A baro-only aiding loop during the coast is a later gate extension.
- The latch correction is bounded by the protection level, since the monitors fire before the spoofer gets further; it is not the size of the attack.

3. Trial KF (PROBATION only)

Step	Action
Probation opens (startTrial, acted on at the start of the next epoch)	trialKF = KF, states and covariance as the 100 Hz side extrapolated them; no propagation on that epoch. The operational KF keeps draining meanwhile (negligible by then)
Start of each PROBATION epoch	trialKF.states = accumPhi * trialKF.states; trialKF.covariance = accumPhi * P * accumPhi' + accumQ, symmetrised (the trial has no 100 Hz extrapolation of its own)
Update	kfUpdate(measurement, trialKF); h(x) must use the states of the struct passed in
After the gate call	trialKF = kfUpdate result (states and covariance). No setKF, no feedback, no drain: the trial has no mechanization of its own and its states must keep the full increments
Veto	trial dropped
Commit	trial xPost, PPost go through the normal setKF as nav.state, nav.covar; trial no longer used

4. What each mode compares, and against what

Mode	Active filter	Test	Compares	Against	Statistic and threshold	Result
NOMINAL, warm-up (info.armed = false)	operational KF, updated normally	none: the monitors arm after ARM_EPOCHS (10) consecutive epochs with numMeas >= REVAL_MIN_MEAS and K_MISSED_DETECTION * sigma_pos(P+) < ARM_PL_MAX (100 m); a PL above the limit restarts the count, a row dropout with the PL inside pauses it; sticky once armed (until a gate re-initialisation)				no windows, no alarms, no latch; the anchor follows the current solution so it is fresh when the monitors start
NOMINAL	operational KF, updated normally	SS, every open window, every monitored axis, every epoch	window separation (increments since the window opened)	that window's INS-only coast	abs(d_axis) > K_FALSE_ALERT * sqrt(P_C - P_KF)	any alarm: LATCH to the anchor
NOMINAL	same	CPI, when a window reaches N = 10	normalised innovation projections on the axis, $\xi = f'S^{-1}y / \sqrt{f'S^{-1}f}$	N(0,1) under no attack	sum(xi^2) > CPI_THRESHOLD (Gamma, 45.64)	any alarm: LATCH; a window alarm-free for its whole life refreshes the anchor
COAST	operational KF not updated (scratch update feeds y, H, R only)	re-validation, every epoch	raw innovation y (all rows)	the INS-only coast (the operational KF's own prior) with its grown covariance	$y'(H P_C H' + R)^{-1}y < \chi^2_{inv}(1 - 1e-3, \text{numMeas})$ and numMeas >= REVAL_MIN_MEAS (4); baro-only epochs cannot pass	10 consecutive passes: PROBATION; a fail resets the dwell. Monitors do not run

Mode	Active filter	Test	Compares	Against	Statistic and threshold	Result
PROBATION	trial KF, updated	SS and CPI on the trial, same as NOMINAL	trial increments (probSep)	the coast ($P_C - P_{\text{trial}}$)	as NOMINAL	any alarm: VETO to COAST (dwell reset); 18 quiet epochs: COMMIT
PROBATION	same	re-validation, diagnostic only	$y - H * (-\Phi * \text{probSep})$	the coast	same statistic, logged as <code>info.qReval</code>	no decision

Why two stages: the COAST test is coarse and its gate widens as the coast covariance grows, so a spoofer resuming with an offset inside that band passes it. PROBATION lets a filter follow the GNSS and runs the full monitor bank on it, with NOMINAL sensitivity, without touching the operational solution. None of these tests reads `KF.states`; the residual shrinking during the coast is the estimate having no information, not a sign that the attack ended.

5. Gate outputs per event

Event	Flags	nav.state	nav.covar	kfCommand
Latch	<code>applyCorrection</code> , <code>eventLatched</code> , <code>eventAnchorEpoch</code>	<code>operational</code> <code>xPost</code> - <code>anchor.separation</code>	<code>anchor.covariance</code>	none
Probation opens	<code>eventProbationStarted</code>	none	none	<code>startTrial = true</code>
Veto	<code>eventProbationVetoed</code>	none	none	none
Commit	<code>applyCorrection</code> , <code>eventHandback</code>	<code>trial</code> <code>xPost</code>	<code>trial</code> <code>PPost</code>	none
Alarm without usable anchor	<code>eventLatched</code> , <code>anchorMissing</code>	none	none	none; mode still COAST
Every epoch	<code>mode</code> , <code>ssAlarm</code> , <code>cpiAlarm</code> , <code>alarmPerAxis</code> , <code>maxProtectionLevel</code> , <code>qReval</code> , <code>dwellCount</code> , <code>coastEpochs</code> , <code>ssRatio</code> (SS margin per axis, alarm > 1), <code>cpiRatio</code> (CPI margin per axis, alarm > 1), <code>armed</code> (monitors active; false during the warm-up after init)		<code>coastCov</code> (diagnostic)	
Fault telemetry	<code>inputFault</code> (non-finite input: epoch dropped, gate re-initialises on the next good epoch), <code>numMeasClamped</code> (host passed more than <code>MAX_MEAS</code> rows), <code>solveFault</code> (an S or residual covariance was not positive definite: that CPI epoch counts <code>xi = 0</code> , re-validation cannot pass)			

After a commit the anchor is the committed solution (separation zero, covariance the trial's `PPost`), the probation windows carry over, and a new attack goes through the same cycle with no limit on the number of cycles.

6. Exception handlers (audit of the runtime path)

Hazard	Where	Handling
$S^{-1} y$, $S^{-1} f$ on the padded S	SPF_cpiMonitor	live-row variances must be positive; padding diagonal filled with the largest live variance so the matrix is non-singular; then the host's <code>matrixInv</code> (flags non-finite / singular input); a flagged epoch is dropped (<code>xi = 0</code>), <code>solveFault</code>
$r' S_r^{-1} r$ on an unusable residual covariance	SPF_revalidation	live-row variances must be positive, then the host's <code>matrixInv</code> ; <code>q</code> must be finite and ≥ 0 to pass; flagged: no pass, <code>solveFault</code>
<code>numMeas > MAX_MEAS</code> slicing fixed buffers	<code>setKfMeas</code> , <code>kfMeasFromUpdate</code> , SPF_cpiMonitor, SPF_revalidation	clamped at ingress and defensively at use; <code>numMeasClamped</code>
NaN / Inf in any input (failed host update, uninitialised <code>propTel</code>)	SPF_gate	<code>isfinite</code> check on every input; epoch dropped with <code>inputFault</code> , state and epoch counter reset, re-init on the next good epoch (mirrors the host's own failed-update reset)
<code>sqrt</code> of a negative variance (<code>P_C - P_KF</code> , <code>P_C</code> , <code>coastCov</code> diagonal)	SPF_ssMonitor, SPF_protectedNav	guarded: test reported undefined (no alarm), sigma 0
division <code>gamma / sqrt(sigma2)</code> with <code>sigma2 = 0</code> (axis unobservable)	SPF_cpiMonitor	guarded: <code>xi = 0</code>
startup anchor stamped with epoch 0 (the "no anchor" sentinel) and propagated once too often	SPF_protectedNav	during the warm-up the anchor is re-seeded every epoch with that epoch's (<code>x+</code> , <code>P+</code>) and stamp, so the first armed epoch starts from a fresh anchor
false latch on an unconverged filter right after (re)initialisation (few rows, large P , interval matrices just reset)	SPF_protectedNav	warm-up arming: no window and no latch until <code>ARM_EPOCHS</code> consecutive epochs qualify (<code>numMeas >= REVAL_MIN_MEAS</code> , <code>PL < ARM_PL_MAX</code>); <code>info.armed</code>
<code>uint32</code> epoch differences, <code>uint8</code> counters and loop variables	all	checked: no wrap possible (<code>anchor.epoch <= epoch</code>), classes consistent
<code>REVAL_THRESHOLD_TABLE(numMeas)</code> index	SPF_revalidation	<code>numMeas</code> clamped to <code>MAX_MEAS</code> (= <code>CST_gnssHybrid.MAX_MEASURES</code>), table has 60 entries and the index is clamped to its length

Covered by `tests/test_error_handlers.m`. On a host update flagged `failed`, pass the epoch as `numMeas = 0`, `xPost = xPrior`, `PPost = PPrior`, or set `resetRequest = true` on that call; a NaN slipping through is caught by the input check.

7. Deferred

- Baro-only aiding during COAST and PROBATION (vertical channel).
- Coast-time budget (Implementation Guide rule 4): each cycle costs a coast and widens the coarse re-validation gate.
- `MAX_ANCHOR_AGE` is a startup-only guard today.
- Warm-up arming uses a fixed protection-level limit (`ARM_PL_MAX`); a convergence test on the filter's own covariance rate could replace it. `ARM_EPOCHS` is 10 in the repository (simulation references); the host runs with 120 (60 s) because its post-reset bias convergence takes that long (see `docs/CONCEPTS.md` section 6). A row dropout pauses the

count; only a PL above `ARM_PL_MAX` restarts it.

- Outage re-arm (disarm after a long GNSS outage so re-acquisition goes through the warm-up): not needed on the jammed data set, where the host's own reset already re-initialises the gate; relevant only for outages that end below the host's reset limit.

Runtime-path review (2026-09-07/08)

Four independent reviewers (paper fidelity, FSM logic, integration hazards, Coder/types) audited the deployed path `spoof_monitor_2hz -> SPF_protectedNav -> SPF_monitorPool -> {SPF_cpiMonitor, SPF_ssMonitor}, SPF_insCoast, SPF_revalidation, STRUCT_SPF, CST_spfParam`. Every finding was then challenged by two independent skeptics; 47 verdicts came back (43 confirmed, 4 refuted, all refuted ones were "info"), 33 verdicts were lost to a session usage limit and are marked *unverified* below.

Confirmed correct (paper lens)

- SPF_cpiMonitor implements Eq. 17/20/29/33/35; per-epoch normalisation makes the H_0 statistic exactly $\text{Gamma}(N/2, 2)$.
- $\Omega = \sigma_2^2 \gamma_u \cdot \sigma_t^2$ is the paper's Eq. 37; Eq. 38 via `gammainc(T_N/(2(1+Omega)), N/2)` reproduces the reference P_{MD} values.
- Baked constants match the offline design to $1e-12$; the per-epoch false-alert budget totals $\leq P_{FA+} + P_{FA-} = 1e-5$.
- SPF_ssMonitor implements Eq. 49-52 with correct time alignment (post-update KF at k vs coast propagated $k-1 \rightarrow k$); window semantics match Fig. 9.
- revalidation's $r'(H P_C H' + R)^{-1} r$ is a valid chi-square(m) test.

Confirmed findings and status

#	Finding	Sev	Status
1	Measurement count frozen at init; any satellite rise/set breaks the pool, CPI and re-validation	blocker	fixed (MAX_MEAS buffers + per-epoch numMeas; <code>tests/test_variable_numMeas.m</code>)
2	Re-validation gate is a single $\chi^2(16)$ threshold; must be $\chi^2(m)$ per satellite count	major	fixed (REVAL_THRESHOLD_TABLE)
3	Re-validation assumes linear $z = H x$; host $h(x)$ is nonlinear	major	fixed (residual = innovation - H (coast - prior))
4	Anchor bring-forward replays every past interval with the current Φ_{acc}/Q_{acc}	major	fixed (anchor kept live, propagated every epoch)
5	$\sigma_{SS} = 0$ turns the SS gate into "alarm on any nonzero separation"	minor	fixed (undefined test -> no alarm)
6	alarmPerAxis indexed by state index in SPF_monitorPool but by bank position in SPF_ssMonitor	major	fixed (bank position everywhere)
7	Struct fields change class at runtime (info.mode, anchor.epoch, eventAnchorEpoch, freeSlot) — Coder rejects	major	fixed
8	anchorMissing branch (alarm before the first clean close) freezes on the spoofed posterior	major	mitigated : during the warm-up the anchor follows the solution, so the first armed epoch has a fresh anchor; flag still only reported on the latch epoch
9	MAX_ANCHOR_AGE can never trigger at a latch (anchor is 1 epoch old); the drift budget it was meant to enforce is a coast-time budget	major	documented ; coast-time budget deferred by the user; <code>info.coastEpochs</code> telemetry added
10	No maximum coast time / dead-reckoning declaration (guide rule 4)	major	deferred by the user until the system is integrated and tested
11	run_recovery tests <code>isnan(t_anchor)</code> but the sentinel is 0	minor	fixed

#	Finding	Sev	Status
12	Coast = $\Phi_{acc} * state$ is the error-state time update, not an INS mechanization; meaning of <code>kfState / reset_state</code> depends on the host's feedback convention	blocker	fixed: engine rewritten in update-increment form ($x+ - x_{bar}$), host owns its filters, gate returns corrections applied through the host's own <code>setKF</code> (see <code>docs/HOST_2HZ_WIRING.m</code>)
13	<code>kfState</code> mirror + Eq. 49 collapse if the host zeroes its error state after feedback	blocker	fixed: the host reduces <code>KF.states</code> by <code>gain*states</code> after feedback (case A); separation is now the accumulated increment <code>d <- Phi d + (x+ - x_bar)</code> , invariant to that bookkeeping
14	<code>reset_state / reseed / feedback</code> semantics only defined for the linear total-state harness	major	fixed: <code>nav.applyCorrection/correction/covar</code> at latch and commit, <code>kfCommand.startTrial =</code> copy the coasting KF; harness (<code>recovery_nav_sim, scheduler_sim</code>) follows the same contract
15	PROBATION trusts the host to apply the reseed and never verifies it	minor	documented (contract in <code>spoof_monitor_2hz</code> header)
16	Host KF left free-running on spoofed GNSS after LATCH/VETO until the next probation reseed	minor	by design (fed architecture); documented
17	Persistent sys/epoch: a host <code>clear functions</code> silently re-arms feedback	major	documented; host must pass <code>resetRequest</code> deliberately
18	SS window runs N-1 tests / coast propagations while CPI uses N innovations	info	by design (CPI-aligned window); documented
19	Anchor is the clean window's INS-only coast rather than the window's final KF solution	info	by design (conservative; the coast never ingested the suspect GNSS)
20	<code>altXCheck</code> struct unused; no independent altitude source	info	left for a later baro cross-check

Unverified (skeptical pass lost to the usage limit)

- PROBATION/handback require the host EKF to run open-loop for 18 epochs and accept a reseed (integration).
- Eq. 50 variance can go negative/~0 if the host's own P propagation differs from `spoofInfo.phiAcc/qAcc` (numerical) — mitigated by #5.
- CPI assumes a joint innovation vector with the full S against the prior (sequential-scalar hosts would need a change).
- NaN/Inf from an ill-posed host S or P are swallowed silently.
- Pool struct (~1.3 MB) copied by value several times per epoch; `obsMatrixBuffer` stores all 60 columns although only the position columns are used.
- "Deterministic rounding" scalar loops in `SPF_cpiMonitor` follow two `mldivide` calls, so they do not buy bit-exactness.
- Coast covariance symmetrised in `SPF_insCoast` but not in `SPF_ssMonitor` (no observable effect).

Gate concepts: separation, coast, anchor, warm-up

Working notes from the design discussion, in question-and-answer form. `docs/HOST_DECISIONS.md` is the contract; this file explains the quantities it uses. Line numbers refer to the current `SPF_protectedNav.m`.

1. Separation and "the coast of the reference"

- **Coast of a reference epoch**: the filter's estimate at that epoch, propagated forward by the INS only (Φ), with no GNSS update. It is what the solution would be today if no measurement had been applied since the reference. Paper E28.
- **Host solution**: the same start plus every GNSS correction accepted since. Each correction is an increment $dx = K*y$ applied at its epoch, and from then on it rides along the INS dynamics like everything else in the state.
- **Separation** = host solution now minus coast of the reference = the sum of the increments since the reference, each propagated from its own epoch to now.

Epoch	Increment $K*y$	Host solution	Coast of reference	Separation
0		2.0	2.0	0
1	+0.5	2.5	2.0	0.5
2	+0.3	2.8	2.0	0.8
3	-0.1	2.7	2.0	0.7

(one axis, $\Phi = 1$ for position; with velocity coupling an increment's contribution keeps changing after it is applied, which is why each one is Φ -propagated)

2. Why the gate keeps the difference, not two states (E28 in increment form)

The paper keeps the filter state and the coast state and subtracts them at the end. The gate keeps only the difference:

- filter: $x_{KF}(k) = \Phi * x_{KF}(k-1) + dx(k)$
- coast (E28): $x_C(k) = \Phi * x_C(k-1)$
- subtract: $d(k) = \Phi * d(k-1) + dx(k)$

This is E28 subtracted from the filter's own recursion: every term of E28 is there, applied to the difference, plus the one term the coast lacks. In a closed-loop error-state filter there is no single absolute state to subtract from (the estimate is split between `stateFB` and `states`, and the split depends on the feedback gain). The increment $K*y$ is the same for any split, so the difference form is the only one that is invariant to how the host stores its estimate (checked by `tests/test_feedback_split.m`). The coast covariance line next to it is E31 unchanged.

3. P_C and P_{KF}

- **P_{KF}** : the host filter's covariance now, P_+ after this epoch's update (`kfMeas.postCov`). Shrinks or holds when measurements arrive.
- **P_C** : the coast covariance, the reference epoch's P_+ propagated with Φ and Q only, never updated. Only grows. Each open window carries its own P_C from the epoch it opened; the anchor carries its own too.

- $P_C - P_{KF}$ is what GNSS has contributed since the reference and, for a consistent linear filter, the covariance of the separation. The SS test uses its diagonal on each monitored axis: $\sigma_{SS} = \sqrt{P_C - P_{KF}}$, alarm when $|d| > k_{FA} * \sigma_{SS}$.

	Variance	Sigma
P_C after 5 s of coast (Q adds 0.2 m ² /s)	5 m ²	2.24 m
P_{KF} after five GNSS updates	1 m ²	1 m
$P_C - P_{KF}$ (separation)	4 m ²	2 m
SS threshold $k_{FA} * \sigma_{SS}$		10.5 m

Consequences: the threshold is not a fixed distance (tolerant under poor geometry, tight under strong GNSS; the test asks whether the pull is plausible, not whether it is big). A zero or negative $P_C - P_{KF}$ on an axis makes the test undefined on that axis: no alarm, no division. In COAST the operational filter is not updated, so P_C and P_{KF} coincide and the coast covariance is the filter's own prior; that is what re-validation compares the raw innovation against.

4. Windows and the anchor

- One window opens every alarm-free armed epoch (line 147) and lives `WINDOW_LENGTH = 10` epochs, so about ten are open at once and one closes every epoch. A window's separation starts at zero at its open epoch and is dropped at its close.
- The anchor uses the same recursion (lines 99 to 101, every armed `NOMINAL` epoch), so it is always "live": subtracting it from today's `x+` gives the anchor's coast at today, nothing to replay.
- **Refresh**: a window that closes with no alarm over its whole life replaces the anchor with its own separation and covariance and stamps it with the close epoch (lines 114 to 119). In clean operation the anchor is replaced every epoch; it is the most recent certified window, not a particular one. Its reference is the opening epoch of that window, so a latch also removes that window's increments: one window length (5 s) more conservative than strictly necessary, deliberately (a below-threshold spoof starting inside the window could have touched its last increments). Anchoring at the close instead is a one-line change (zero the separation at the refresh, keep the window's `P+`).
- **Warm-up** (unarmed): re-seeded every epoch with separation zero, covariance `P+`, stamp now (lines 87 to 91).
- **Commit** after probation: separation zero, covariance the trial's `P+`, stamp the commit epoch (lines 245 to 249).
- **COAST and PROBATION**: not propagated; at the latch its covariance became the coast covariance and the operational filter's own prior carries it.
- **Age limit**: checked only at an alarm (line 124). If the anchor is older than `MAX_ANCHOR_AGE` or not valid (lines 135 to 138): no correction, `info.anchorMissing`, COAST from the current (possibly contaminated) solution with the current `P+` as coast covariance. No new anchor until the next commit. In practice hard to reach now: the age grows only while no window closes clean, and in `NOMINAL` that means alarms, which already latch.

5. Warm-up arming

Code: `SPF_protectedNav.m` lines 66 to 85 (counter), 87 to 91 (unarmed action), constants `CST_spfParam.m` lines 71 to 73, telemetry `info.armed`.

- After a gate (re)initialisation the host filter is not converged: initial `P`, zero error state (the receiver clock included), few rows, interval matrices just reset. Its first corrections are far larger than its own covariance predicts, and the SS test reads that as a pull.
- Clearing the persistent data does not avoid this; it causes it. "Cleared" and "converged" are different states.

- In a full run the transient happens during alignment, when the gate is inactive (`navActive = false`), so the first active epoch sees a settled filter. A segment started mid-flight with a manual reset, or an emergency reset, starts filter and gate together on a cold filter.
- The monitors therefore arm only after `ARM_EPOCHS` qualifying epochs, with `numMeas >= REVAL_MIN_MEAS` and `K_MISSED_DETECTION * sigma_pos(P+) < ARM_PL_MAX` (100 m). A PL above the limit restarts the count (the filter is diverging or coasting); a row dropout with the PL still inside only pauses it. Arming is sticky until the next re-initialisation. The simulation references use 10 epochs; the host uses 120 (60 s), see section 6.
- Ten epochs is a count, not a proof of convergence; the PL condition is the real guard. In the mode figure the warm-up is the grey band, which must cover the whole transient; if the SS margins are still near 1 when the band ends, raise `ARM_EPOCHS` or lower `ARM_PL_MAX`.
- Any reset that sends the KF back to its initial P (emergency reset, re-alignment) must reset the gate too, through `firstCall` or by dropping `navActive` during the reset. Otherwise the anchor, windows and trial carry pre-reset history against a restarted filter and a latch is almost guaranteed.

6. What the jammed data set showed (470 s GNSS outage, host reset at re-acquisition)

- The host's protective reset fires when GNSS returns and the position differs from the INS by more than its limit; it re-seeds the filter with the initial P. Everything the gate alarmed on in the first runs was the convergence after that reset (clock, velocity and biases estimated from scratch): SS margins of 5, CPI 20, NIS 33 on the first post-reset epoch.
- The covariance during the outage itself was honest: `sigma_N` from `P+` was 11.7 km against a true drift of 22 km (ratio 1.9), `sigma_E` 11.7 km against 2.4 km. Q is not the problem on this data; the earlier reading that P had not grown enough was wrong, it was the post-reset P.
- Two things fixed it: the gate is reset together with the host (`navActive` dropped in the reset and alignment modes), and the warm-up is long enough for the post-reset bias convergence, `ARM_EPOCHS = 120` (60 s) in the host configuration instead of the simulation default of
- With both, the non-shadow run matches the shadow run: one host reset, no latch, errors at zero after re-acquisition.
- Side effect of the first arming rule: an epoch with fewer than `REVAL_MIN_MEAS` rows restarted the count, so on data with early dropouts the first warm-up lasted 150 s instead of 60 s. The rule now pauses the count on a row dropout and restarts it only when the PL leaves the limit.
- After the reset the filter is still settling 110 s later: a short GNSS gap at 970 s gave SS margins of 0.85. Not an alarm, but the margin to watch on runs that start from a reset.